You said:

could you please create a 30 days plan for terraform as industry standard

Worked for 40s

ChatGPT said:

30-Day Terraform Roadmap – Industry Standard (AWS + DevOps)

Goal: Learn Terraform from fundamentals to production-level Infrastructure as Code (IaC), including AWS infrastructure provisioning, reusable modules, environment management, remote state, and automated deployment through CI/CD pipelines.

This plan is designed around the way Terraform is used in real DevOps and Cloud Infrastructure projects, rather than simply learning Terraform commands.

Recommended commitment: 2–3 hours daily.

Your 30-day learning journey

Week 1 Foundation

Terraform fundamentals + AWS basics

Week 2: Modules, state management and environments

Week 3: Real-world AWS architecture using Terraform

Week 4: CI/CD, security, testing and production practices

Week 1: Terraform Fundamentals (Days 1–7)

Target: Understand how Terraform works and independently provision basic AWS resources.

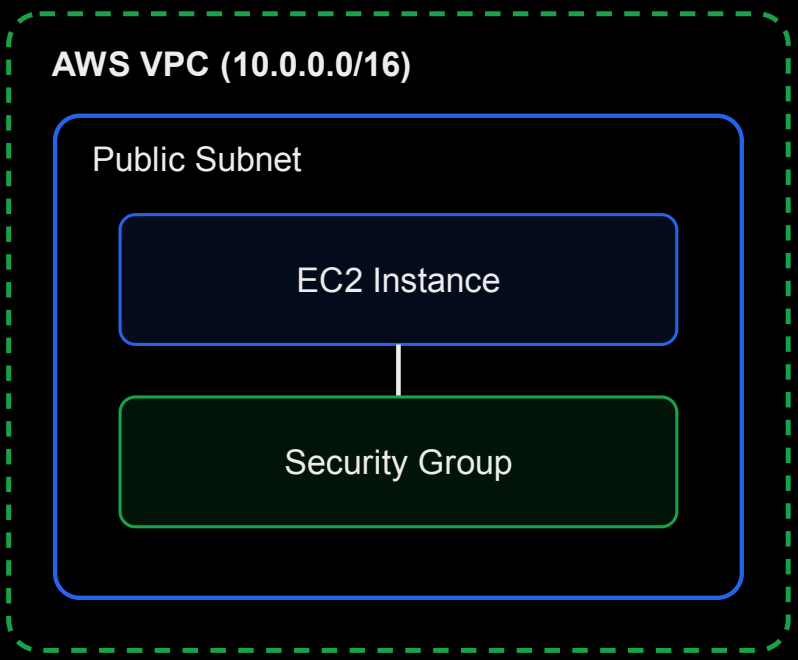
Day	Topics to learn	Hands-on assignment
1	Introduction to IaC, Terraform architecture, providers, HCL	Install Terraform, AWS CLI and VS Code
2	Providers, resources, Terraform configuration	Create your first AWS EC2 instance
3	`init`, `plan`, `apply`, `destroy`, `fmt`, `validate`	Practice the full Terraform lifecycle
4	Input variables, outputs, data types and locals	Dynamically configure an EC2 instance
5	Data sources, references and dependencies	Retrieve existing AWS VPC and AMI details
6	`count`, `for_each`, conditional expressions and dynamic blocks	Create multiple EC2 instances using `for_each`
7	Revision and mini-project	Deploy EC2, VPC, subnet and Security Group together

Week 1 mini-project

Create the following infrastructure entirely through Terraform:

Project 1: Basic AWS Infrastructure

Ask ChatGPT



Configure an Internet Gateway and appropriate routing so the public instance is reachable as intended.

By Day 7, you should be comfortable explaining what Terraform creates, how it tracks resources, and what happens when you modify your configuration.

Week 2: Production Terraform Structure (Days 8–14)

Target: Learn how companies organize Terraform code to support multiple environments and teams.

Day	Topics to learn	Hands-on assignment
8	Terraform state, state files, drift and locking	Inspect Terraform state using CLI
9	Remote backend, S3, state locking, versioning	Configure an S3 remote backend
10	Terraform modules: root and child modules	Create reusable EC2 and VPC modules
11	Module inputs, outputs, versioning and dependencies	Connect multiple modules
12	Environment management using tfvars and separate state	Create NPR and PRD environments
13	Resource import, `import` blocks, `moved` blocks, `removed` blocks	Import an existing AWS resource
14	Revision + project	Build a reusable multi-environment Terraform repository

Important industry practice: Use separate state for production and non-production. For modern S3 backends, use `use_lockfile = true` and enable bucket versioning. DynamoDB-based state locking is now deprecated.

HashiCorp ... +1

Recommended project structure

By Day 14, your Terraform repository should resemble:

Plain text

```
terraform-aws-infrastructure/
|
├── modules/
|   ├── vpc/
|   |   ├── main.tf
|   |   ├── variables.tf
```

```
│   │   └── outputs.tf
│   │
│   ├── ec2/
│   ├── security-group/
│   └── s3/
│
├── environments/
│   ├── npr/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   └── terraform.tfvars
│   │
│   └── prd/
│       ├── main.tf
```

Both NPR and PRD reuse the same modules, while having isolated root configurations and state.

This also allows you to perform production-only imports without accidentally importing the same resources into the non-production environment.

Week 3: Real AWS Infrastructure Project (Days 15–21)

Target: Build the kind of AWS infrastructure that a DevOps engineer might manage in an actual company.

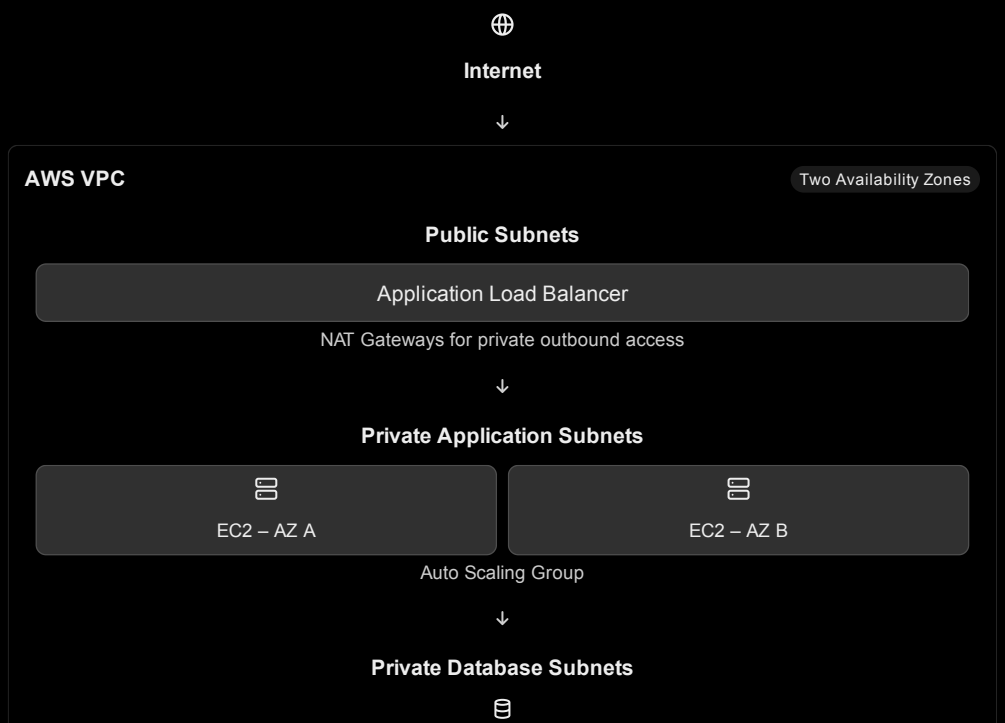
Day	Topics to learn	Hands-on assignment
15	VPC architecture, CIDR, subnets and route tables	Create a custom VPC
16	Internet Gateway, NAT Gateway, public/private subnets	Build a multi-tier network
17	Security Groups, IAM roles and policies	Configure access using least privilege
18	ALB, target groups, listeners and EC2	Deploy infrastructure for a web application
19	Launch templates and Auto Scaling Groups	Implement an autoscaling EC2 architecture
20	RDS, subnet groups and networking	Add a private PostgreSQL database
21	End-to-end integration	Complete an AWS three-tier infrastructure project

Week 3 major project

Create the following architecture using reusable Terraform modules.

Project 2: Three-Tier AWS Architecture

A simplified logical view of the architecture.



Amazon RDS

PostgreSQL database

Expected learning outcome: You should understand the relationship between Terraform code, AWS networking, compute infrastructure, database connectivity, module dependencies and deployment order.

For learning purposes, use small resources and destroy them after exercises. NAT Gateways, load balancers and RDS can generate charges even with little traffic.

Week 4: Terraform CI/CD and Production Practices (Days 22–30)

Target: Automate Terraform deployments using GitHub Actions and implement the quality, security and approval controls commonly expected in enterprise projects.

Day	Topics to learn	Hands-on assignment
22	Git fundamentals, branching and pull requests	Push Terraform project to GitHub
23	GitHub Actions fundamentals	Create your first Terraform workflow
24	Automated validation	Implement `fmt`, `init`, `validate` and `plan`
25	AWS OIDC authentication	Configure GitHub Actions to assume an AWS IAM role
26	Multi-environment CI/CD	Separate NPR and PRD deployment workflows
27	Approval gates and controlled deployment	Implement production approval before apply
28	Security scanning and Terraform testing	Integrate Checkov or Trivy and `terraform test`
29	Drift detection, troubleshooting and recovery	Practice detecting changes made outside Terraform
30	Final capstone project	Demonstrate a complete infrastructure deployment using CI/CD

HashiCorp recommends maintaining the dependency lock file, pinning versions, protecting credentials and incorporating Terraform module tests into automated workflows.

HashiCorp ... +1

Your production-style CI/CD pipeline

By the end of Day 30, aim to implement this workflow:

Terraform Deployment Pipeline

Developer creates Pull Request

Terraform infrastructure changes

↓

CI Validation

terraform fmt → validate → security scan

↓

Terraform Plan

Review proposed infrastructure changes

↓

Code Review

PR reviewed and merged

↓

NPR Deployment

Apply using non-production state

↓

Production Approval
Manual approval + environment protection

↓

PRD Deployment
Apply the reviewed production plan

NPR and PRD should have separate AWS permissions and Terraform state. Production deployment should also verify that the reviewed plan is the one being applied.

Final project: What you should build by Day 30

Your GitHub repository should demonstrate the following:

Terraform AWS Capstone Portfolio project

Infrastructure requirements

0 of 13 completed 0%

- [ ] Reusable Terraform modules
- [ ] Custom AWS VPC with public/private subnets
- [ ] Internet Gateway and NAT Gateway
- [ ] Security Groups and IAM roles
- [ ] EC2 and Auto Scaling Group
- [ ] Application Load Balancer
- [ ] Private RDS database
- [ ] Separate NPR and PRD environments
- [ ] S3 remote state with locking
- [ ] GitHub Actions CI/CD pipeline
- [ ] Automated security and validation checks
- [ ] Production deployment approval
- [ ] README with architecture and deployment steps

Use this checklist to track your capstone implementation.

Recommended learning resources

Use official documentation as your primary reference rather than relying entirely on video tutorials.

1. HashiCorp Terraform Tutorials
Best for fundamentals, modules, state, testing and Terraform CLI.
Official Terraform Tutorials ↗

2. AWS Terraform Best Practices
Understand recommended Terraform architecture and implementation patterns.
AWS Prescriptive Guidance ↗

---

3. GitHub Actions Documentation
Useful for pipeline automation, approval gates and AWS OIDC integration.
GitHub Actions Documentation ↗

Daily study routine

To get the maximum benefit from these 30 days, follow a consistent routine:

Activity	Daily time
Learn theoretical concepts	30 minutes
Follow practical examples	30 minutes
Write Terraform independently	60 minutes
Troubleshoot and document learning	30 minutes
Total	2.5 hours

One rule: Don't just copy Terraform code from tutorials. Write it independently, execute the plan, inspect what AWS actually creates, and understand what happens when you modify or destroy a resource.

What you should know after 30 days

By completing this roadmap, you should be able to confidently explain and demonstrate:

- How Terraform provisions and manages AWS resources.
- How companies organize Terraform code using reusable modules.
- How to deploy the same infrastructure code into NPR and PRD safely.
- How Terraform state, locking, import and drift detection work.
- How to build a CI/CD pipeline with validation, planning and controlled deployment.
- How to troubleshoot failed Terraform deployments.

My recommendation: Treat Days 1–14 as your foundation and Days 15–30 as your practical project. The final GitHub repository will be more valuable than simply having completed a course.

Thirty days can give you a strong hands-on foundation, although becoming proficient with production infrastructure will require continued project experience.

You said:

could you please suggest some couese than can help me to complete all the topic throghly which you have mention abobe

Worked for 21s

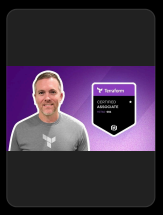
ChatGPT said:

For your 30-day Terraform roadmap, I recommend **combining two Udemy courses with HashiCorp's official hands-on labs**.

One course can teach Terraform thoroughly, while the second can show how Terraform is actually integrated with AWS and CI/CD pipelines in a real DevOps project.

I checked the available courses, including their latest listed updates.

1. Best Terraform courses for your roadmap



My first choice

HashiCorp Certified: Terraform Associate 2026 ↗

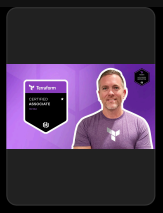
Instructor: Zeal Vora

☆ **4.7/5** · 25.5 hours

Updated October 2026.

This should be your primary learning course. It covers Terraform fundamentals, AWS, state management, modules, expressions and practical exercises.

vanguard.ud… +1



HashiCorp Certified Terraform Associate 004 Exam Prep ↗

Instructor: Bryan Krausen

4.8/5 · 23.5 hours

Updated September 2026.

An excellent alternative to Zeal Vora's course, particularly for structured learning, labs, quizzes and understanding Terraform concepts thoroughly.

Udemy +1



Real-world project

DevSecOps Real-World Project: Build and Operate CI/CD on EKS ↗

Instructor: AR Shankar (Valaxy Technologies)

Approximately 9 hours · Updated September 2026

Particularly useful for learning:

- Terraform AWS modules
- S3 remote backend
- GitHub Actions pipelines
- Terraform plan on pull requests
- Manual approval gates
- DevSecOps practices

It also covers EKS, Docker and GitOps, so only selected sections are necessary for this roadmap.

Udemy +1

Which should you purchase?

I would choose Zeal Vora's course first and AR Shankar's project course second. You don't need to purchase both Zeal Vora's and Bryan Krausen's courses because their foundational content overlaps considerably.

---

2. How to combine these courses over 30 days

Rather than completing one course and only then starting another, follow this mapping.

Your roadmap	Learning resource
Days 1–7: Terraform fundamentals	Zeal Vora
Days 8–10: State and modules	Zeal Vora + HashiCorp labs
Days 11–14: NPR/PRD and resource imports	HashiCorp official tutorials + own project
Days 15–21: AWS architecture	Zeal Vora + independent AWS capstone
Days 22–27: Terraform CI/CD	AR Shankar
Days 28–30: Testing, security, drift and final project	HashiCorp tutorials + AR Shankar

3. Free resources you should not skip

These will fill gaps that certification-focused courses may not cover in sufficient depth.

HashiCorp official practical labs

Terraform Modules

Learn local modules, registry modules, inputs, outputs and environment separation.

Open

Terraform State Management

Resource imports, drift, state manipulation and recovery.

Open

Terraform Automation

Production automation and CI/CD workflows. Open

4. Topics that deserve extra attention

Even after completing these courses, I strongly recommend practising the following independently:

1. Separate NPR/PRD environments and state files.
2. Production-only resource imports using `import` blocks.
3. Creating reusable VPC, EC2 and RDS modules.
4. S3 state locking using the current locking mechanism.
5. AWS authentication through GitHub Actions OIDC.
6. Terraform plan, approval and apply workflows.
7. Handling infrastructure drift and failed deployments.

These are especially important when transitioning from following Terraform tutorials to managing infrastructure in an actual company.

My suggested purchase decision

Budget	What to choose
One course	Zeal Vora
Two courses	Zeal Vora + AR Shankar
No budget	HashiCorp official tutorials

Start with Zeal Vora's Terraform Associate 2026 course. Complete its exercises alongside the first two weeks of the roadmap, and gradually transform that knowledge into your own AWS Terraform repository.

Your objective should be to finish Day 30 with a working Terraform GitHub repository and deployment pipeline—not merely two Udemy completion certificates.



Coursera Ad
Build Skills for a New Career
Explore courses to support your career change.
Learn more

Ads do not influence the answers you get from ChatGPT. Your chats stay private. Learn about ads and personalization